

Page-record granularity

Introduction

数据库系统中，缓存在 dram 中的缓存、索引、以及元数据结构会对数据库的性能产生较大的影响。在传统的 on-disk 数据库设计中，Buffer Pool 的缓存粒度为 Page，其根本的原因是需要和磁盘进行 I/O 交互。细粒度的缓存结构不够 I/O Efficient，导致 dram 和 ssd 之间的磁盘访问速度和效率成为性能瓶颈。

但同时，数据库中的大量工作负载呈现出倾斜分布，例如 zipfian/pareto 分布等，扩展内存对于提升具有倾斜分布的访问模式效果显著。然而基于 page 粒度的访问，特别是点查等具有倾斜访问的工作负载，经常出现一个 hot record 把 4 KB 的 page 锁定在 dram 中，导致该 page 中剩余的 cold record 浪费了宝贵的内存资源。随着我们引入新一级的内存层次，cxl memory 为解决缓存粒度和访问模式之间的矛盾提供了新的契机。cxl memory 可以作为 ssd 之前的一级存储层次，我们可以把数据库中的页面交换过程，从 dram - ssd 之间移动到 cxl - ssd 之间，从而 dram - cxl 之间的访问延迟显著优于 dram - ssd 之间的数据访问。在引入 cxl memory 作为扩展内存的新一级存储层次之后，结合两种不同粒度缓存和索引结构的优势，在 cxl memory 和 ssd 之间以 page 粒度进行页面交换，同时 dram 和 cxl memory 之间以 record 为粒度进行数据迁移。

Background

对于 cxl 接入的场景问题。

1. 单机数据库 + cxl 需要先定位清楚场景，比如为何单机数据库要上 cxl

从扩展内存，加速倾斜工作负载的执行和性能角度

2. 加 cxl 和直接加 ddr 内存有什么不同，单机上现在也能加 5 - 10 TB 的 ddr 内存，加 cxl 内存有成本优势吗？

可以利用其他服务器上的空闲内存，这部分空闲内存非常可观，占云服务中50%服务器上接近50%的内存容量，节省成本

3. 直接透明使用 cxl 内存是否可行，哪里有性能瓶颈

不太可行，存在一定的延迟开销

4. 都连接其他服务器的空闲内存了，这还是单机场景吗？

是的，目前只考虑一个计算节点，不参与其他服务器上的计算。

5. 单机上研究 cxl 目前主要是做异构硬件的协同，大多数不会纠结性能问题，比如 GPU 和 CPU 对于 cache 一致性的访问问题，用 CXL 扩展 GPU 显存等

用了其他服务的内存，那其他服务器要用内存怎么办，业内是否有参考案例。如果没有参考案例，这种集群架构的设计是否合理呢？

设想的是仿照 ASPLOS Pond 中的结构，单机看成一个计算节点，访问 cxl 内存池里的内存。Pond 里面维护了一个 Pond Manager，把 cxl 中的内存资源按照 1 GB 的粒度分片划分给不同的 VM，同一时刻如果服务器要用内存，会执行一个动态迁移的策略，本地异步回收内存资源分配给远端。计算和内存分离，计算节点本地的 dram 比较少，cxl 内存池提供大的内存资源。

现在想到一个应用场景：单机数据库 + cxl 应用在英雄联盟或者 dota 比赛期间的观战服。

（单机场景暂时还没有想到，但是静态扩展 CXL 内存的场景可以描述如下：

比如公司的智能阅卷业务：错峰内存调度

数据库系统需要服务多市中小学期中，期末阅卷业务。同时由于各地期中、期末的时间可能相差 1 - 2 周左右，在这期间完全可以将其他机器上的空闲内存暂时提供给本市的学校，之后再回收。

单机 CXL 扩展内存，利用 record 粒度缓存降低倾斜工作负载 IO 放大的多级缓存结构，场景是：在数据库集群服务中，存在这样一种智能阅卷业务，其中各地学校的期中期末考试存在 1 - 2 周的错峰，这期间完全可以用较为空闲机器上的 DRAM 资源提供给本地的服务器节点，从而降低使用成本。同时，由于 CXL 扩展内存和本地 DRAM 设备存在延迟，带宽上的差异。需要尽可能把最核心的数据存储 DRAM 中，应对 OLTP 的查询业务。而基于 page 粒度的缓存无法支持全部的 page 都是 hot record，因此我们提出 record-page granularity 的多级缓存结构，尽可能保证扩展内存降低成本之后，OLTP 业务的运行性能不下降。）

Motivation

传统物理机架构下，由于内存插槽的限制，使得内存资源扩容难且成本高。CXL（Compute Express Link）技术的出现，使得可以将高性能内存池连接到计算节点，且充分利用其他物理机上的空闲内存资源（如部署计算密集型业务的物理机），支持秒级加载 CXL 内存，将单节点内存扩展至 TB 级别，并且，CXL 内存的单位 GB 成本远低于本地内存（DRAM）。因此，对于数据库系统，可将 CXL 内存作为物理机本地 DRAM 内存的扩展，以实现数据库内存资源的按需独立弹性伸缩，且降低数据库内存总成本。然而，CXL 内存相比于本地 DRAM 内存，在读写延迟、带宽等方面还存在性能差距【Tigon】【HybridTier】【ICGMM】【1-6】【其他参考文献】，如果所有数据都在 CXL 内存中，系统性能会受到影响。

综上，当数据库系统引入 CXL 内存后，如何设计基于本地 DRAM 内存和 CXL 内存的多级缓存架构，以及如何配置本地 DRAM 内存和 CXL 内存的比例，在降低数据库内存总成本的同时提升数据库系统整体性能？

另一方面，为了保证高效的磁盘 I/O 效率，数据库 Buffer Pool 的缓存粒度为 Page；而数据库实际业务的工作负载往往存在数据访问热点，且热点数据往往是 record 粒度的【HybridTier】【Tiered-Index】【其他参考文献】。

上述业务负载特征为基于 CXL 内存的数据库多级缓存设计提供了一个机会：将 record 粒度的

少量极热数据缓存在本地DRAM，以保证热数据访问的低延迟；将page粒度的次热数据缓存在CXL内存，以兼顾性能与存储成本。

但需解决以下关键挑战：1. 本地DRAM中需缓存哪些数据，以及缓存结构、缓存管理策略如何设计？2. CXL内存中缓存结构、缓存管理策略如何设计？3. 如何保证 本地DRAM与CXL内存中缓存数据的一致性？4. 如何设计低开销、且精准的热数据识别机制？...？

【1】 A CXL-Powered Database System: Opportunities and Challenges, ICDE'24

【2】 Bandwidth Expansion via CXL: A Pathway to Accelerating In-Memory Analytical Processing, VLDB'24

【3】 An Examination of CXL Memory Use Cases for In-Memory Database Management Systems using SAP HANA, VLDB'24

【4】 Tigon: A Distributed Database for a CXL Pod, OSDI'25

【5】 Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices, MICRO 2023

【6】 Memory Sharing with CXL: Hardware and Software Design Approaches, HCDS 2024

Design principle

云原生数据库基于共享存储，设计上类似于传统的，带有磁盘的 database design。但是现在有了 cxl，相当于拥有了非常大的扩展内存空间。(这个依据是微软的数据 Redy: 50% 的服务器可以通过一个 switch 获得 1 TB 的额外内存收益，通过三个 switch 获得 30 TB 的额外内存收益，通过五个 switch 获得 100 TB 的额外内存收益)。所以引入 cxl 之后，云原生数据库在某种程度上可以逼近 in-memory database. 因此很多设计优化可以从 in-memory database 的优化中借鉴迁移。比如 in-memory database 的特点是，没有页面到磁盘的换入换出，页面的物理内存地址在整个运行周期中保持不变，所以有 pointer-swizzling 这些在初始化加载时就能完成的页表转换。

同时，云上数据库依然存在 larger-than-memory 的工作负载，所以需要保留块粒度存储换入换出的设计，在逼近 In-memory 的同时，兼顾 larger-than-memory 的需求。

keyword

相关工作检索过程中的关键词:

1. fine-grained caching & database
2. record-level caching & buffer pool
3. multi-granularity caching & disaggregated memory
4. cxl & buffer management
5. cxl & index structure & B+Tree & CXL memory
6. record-oriented caching/tuple-oriented caching
7. sub-page caching

Related Work & design

Design insight

直接对标的工作是周信静的 2-Tree 和索引层面 record 粒度迁移存在的问题。第一个是 record 在索引上的迁移存在写放大（基于 LSM-Tree 的索引无法进行 in-place update，所以即使是热记录的读取，也需要重新走一遍写的流程。）以及并没有考虑到索引本身的父子结构关系，直接将 B-Tree 分成冷热两棵，父子结构关系一定程度上被破坏。

之前 Three-Tier-BufferManagement (SplitFire) 的工作里提出了 mini-page 和升级到 full-page 的机制，除了 PMem 的场景，需要陈述工作的区别。(TODO)

Design Question

需要解决的挑战:

元数据管理的一致性开销和锁开销

1.考虑 trade off, 比如用了 record 粒度的索引，在 table scan 的时候是扫 page 还是扫 record cache。

如果只需要扫 page, 则表明 record 粒度的 cache 和 page 中的数据需要时时一致的，维护成本也需要考虑，

比如原本更新一行只需要更新 page，现在需要同时更新 page 和 record cache 了。

如果只是更新 record 粒度的cache，则怎么保证 cache 和 page 的一致性也是问题，

尤其是在 table scan 的时候，每次从 page 中扫描一个记录就需要在 record cache 查找其是否已经被更新了。

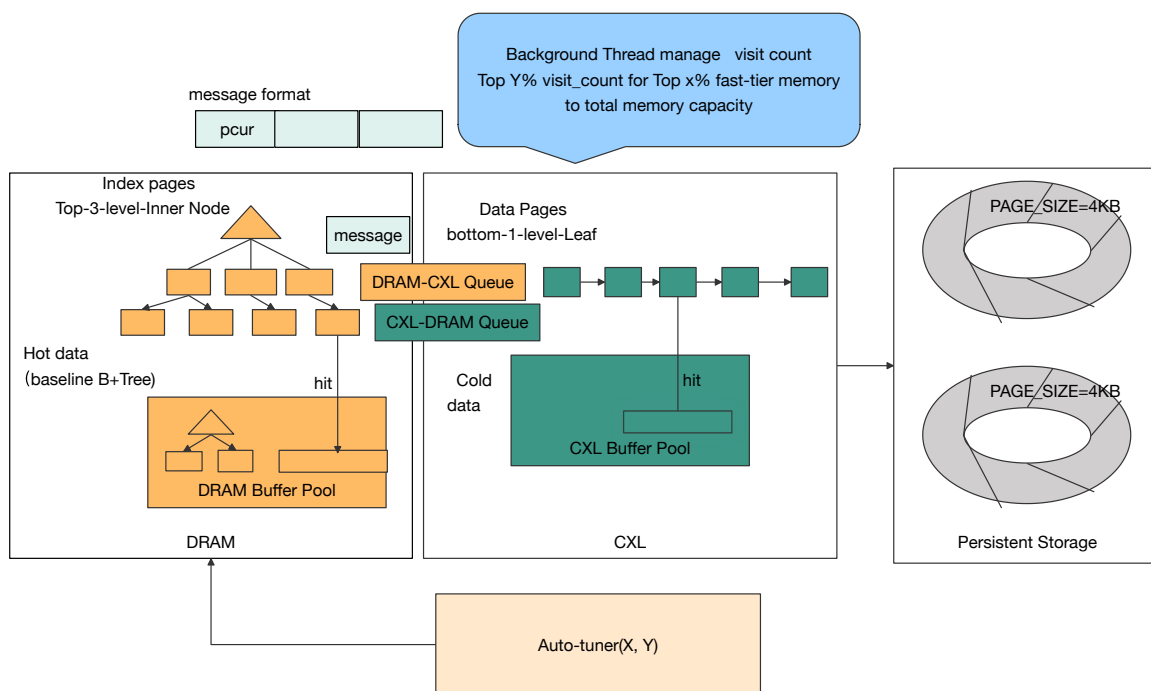
(在数据库场景下 cache trashing 的污染来源: 比如 table-scan 造成的热数据驱逐，page 替换粒度过大导致的 lukewarm 挤占缓存空间

同时数据库场景下的元数据开销来源: 不同粒度数据之间的一致性，元数据的锁🔒消耗 cpu 资源(做到什么程度的元数据和数据分离))

推广（后续工作的点）：

cxl 作为共享内存，dram是每个节点独有的。对于 multi-master 的场景，如果 a 节点改了某个 page, b 节点怎么更新其本地的 dram 上的 record cache

Design



1. 最大的框架，我们分成 DRAM-BP 和 CXL-BP，其中 DRAM-BP 的设计原则是 resource efficient, 提高 DRAM 资源的利用率，尽可能以 record 粒度缓存数据。（借鉴 in-memory database 的设计）

在引入 cxl memory 作为中间层次之后，所有的磁盘交互工作通过 cxl memory 和 ssd 完成。

2. 比较 DRAM-BP 和 CXL-BP 中 Hot index 和 Cold index

周信静的做法是直接分成 hot-tier 和 cold-tier，按照数据访问的频率进行划分标准。但是经过实验测试(命中率日志追踪)，B+-tree 上层的页面总是查询过程中访问的热点，这个查询访问的冷热并不完全是结构上独立的，而是依然具有类似 B+-tree 的父子关系。

我们的设计认为，在 DRAM 中保留 B+-tree 搜索的根页面和内部节点，CXL 中的底层页面才是经常发生 random Disk IO 的随机叶子页面。

💡 TODO: 发现在整个索引结构的访问过程中，根页面和内部页面占的内存空间小，估算 4 层的 B+Tree, 根页面占 1 Page，第一层占 1000 个 Page，第二层占 1000000 个 Page。比较合理的情况大概是所有索引的页面都缓存在 DRAM 里，record 粒度缓存更多地是针对在 CXL 上的随机叶子页面建立。

这样看来, DRAM 里的索引页全部按照 page 粒度, 数据页(从 CXL 随机叶子页面中提取的 hot-record)按照 record 粒度。属于混合缓存。

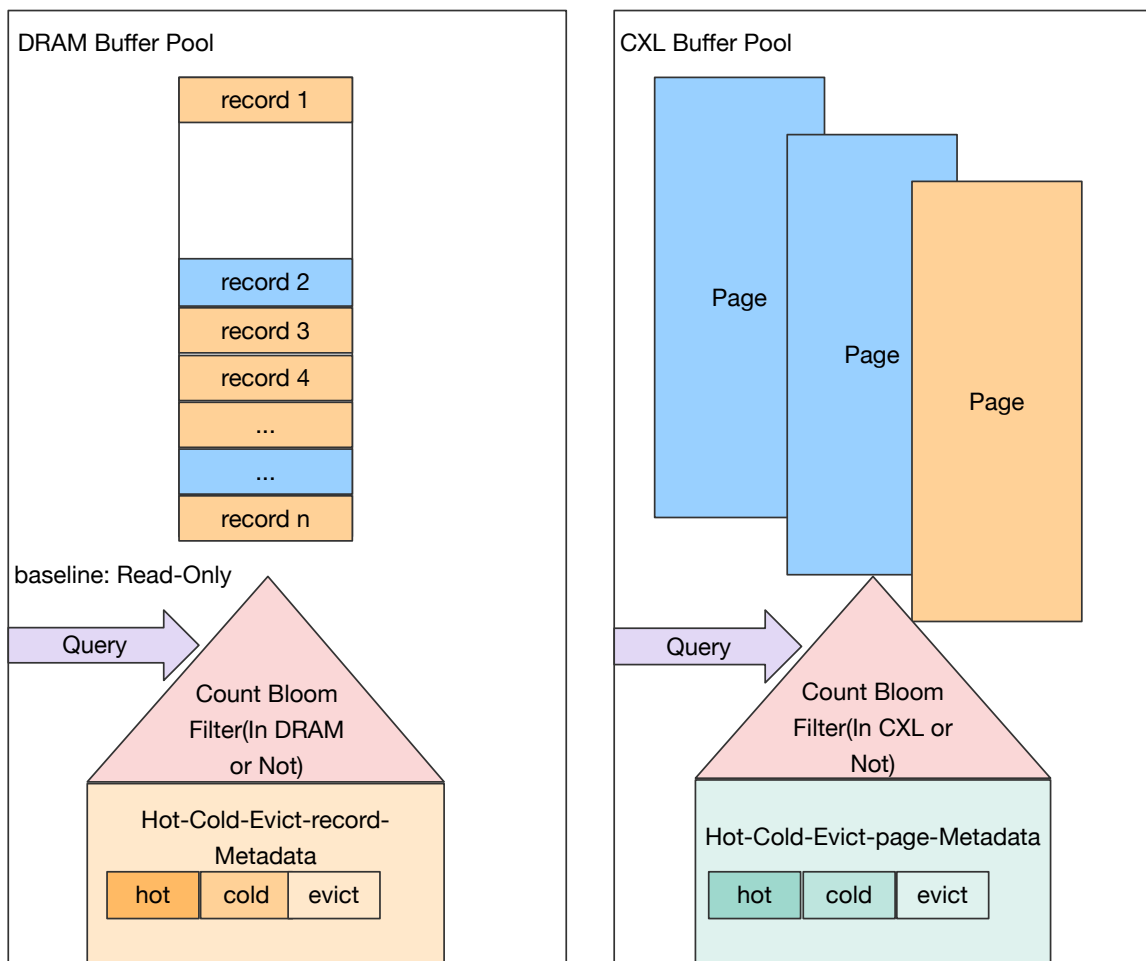
这种设计里: DRAM(Page-sized index + Record-sized data) + CXL(Page-sized data)

这种设计思路和 MySQL InnoDB 里 Adaptive Hash Index 的区别。

冷热数据的交互过程可以通过 DRAM-CXL 和 CXL-DRAM 双向队列完成, 同时 InnoDB 中对 B+Tree 搜索路径优化的 persistent cursor(持久游标, 记录上一次查询到的B+-tree)位置, 可以被改造, 作为 DRAM-CXL 双向队列中传递的一个消息格式。

3. 需要 well-designed 的部分是队列中传递的消息格式, 以及 record 粒度的元数据管理
4. tiered-metadata 的设计, 分层次的元数据通过 Bloom filter 先进行一次过滤
5. 在场景确定之后, 由于 cxl-attached memory 可以扩大, 同时随着数据库运行过程中, fast hot tier vs large capacity tier 的比例在不断变化。

如果 fast hot tier / large capacity tier = x%, 那么就需要识别出当前 working set 中访问最频繁的 top x%数据, 这些数据并不是恒定按照某一个(小于某个固定的 visit_count = Y 的集合就一直占系统总访问数据量的 x%) 后台线程需要维护一个动态变化的 visit_count 转换阈值。

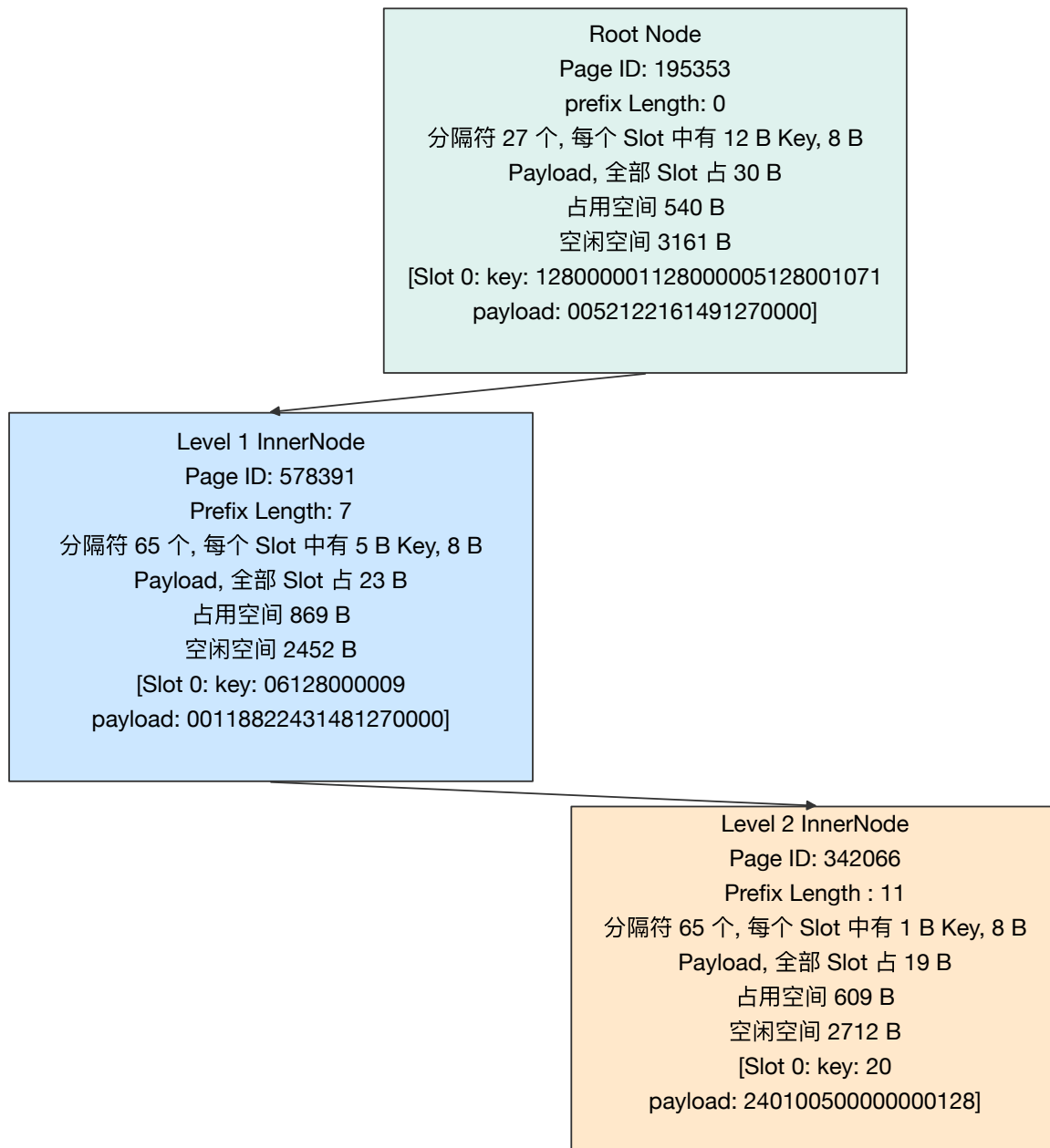


Design Boundaries

传统的OLTP存储引擎以 B+ Tree 为索引，一般 B+ Tree 呈现出“胖树”结构，分为 3 层内部节点和最底层的随机叶子 IO 页面。我们认为 record 粒度缓存的设计边界是，只针对 B+ Tree 索引结构的最后一层数据页进行 record 粒度缓存设计，保留 Page 结构的内部索引。

(TODO: 后续不确定要不要分析 Level 2 的 Page 和冷热 record 的访问均匀程度，比如证明 Level 2 Page 中的 record 访问总体上是 uniform 的 ?)。

打印内部节点的布局 Layout



总结规律:

1. 首先 Slot 本身的元数据信息, 比如 offset, key_length, payload_length 固定占 10 B, 所以在算每个 record 占据空间的大小都加上 10 B 的 Slot[slot_id] 元信息。
2. 可以看到逐层的 key length 递减, 12 B -> 5 B -> 1 B, payload 都是 8 B 的指针
3. 每个 Slot 分配较紧密, 但 page 没有用满。

前 3 层索引页不适合搞 record 粒度缓存的原因:

- Root 和 Level 1 : 访问频率高

- Level 2: Page 数量多了，不一定都缓存在内存里，但需要通过二分查找进行路由，record 粒度缓存会破坏连续内存布局。同时整个record (1 B key 和 8 Byte 指针) 总共 9 Byte 大小，拆分之后元数据管理开销过大。现在的 LeanStore 已经用了 10 B 的 Slot 元数据来管理这里 9 Byte 的 record。

Design Details

按照上次讨论反馈之后的结果，需要分阶段解决三个设计上的核心问题，和一个代码上的问题:

1. hot-page 和 hot-record 的识别。如何从磁盘中快速识别 hot page 迁移到 cxl, 以及如何从 cxl 中的 hot page 快速识别 hot record, 迁移到 dram。(这个过程的问题: 快速识别, 以及应对可能 5 min 变化一次的热点分布)
 - a. 静态的阈值不合适,因为无法确定 Top x% 容量对应的一定是某个固定值的 visit_count (比如 visit_count = 1, visit_count = 2).
2. dram 中使用 record 粒度缓存的同步问题, 比如 table scan 过程中, 如果默认只扫 page, 那么怎么和 record 保持一致? 一致性问题(采用什么样的同步策略? 两阶段同步?)
3. record 粒度缓存带来的元数据和碎片🗑️开销。
4. 代码层面的问题: 之前还是没有彻底理解 buffer pool 和 index 之间的关系, 比如周信静工作中声称: 他们可以完全把 page 粒度的 replacement 和索引的 record-migration 解耦开。从 LeanStore 的源码中理解这个过程。

Implementation Questions

这里记录所有设计、实现过程中遇到的问题。(包括但不限于: 场景适配, 设计优化, cxl 使用情况, os系统调用封装)。

1. 构建 leanstore 过程中编译问题汇总(10 个依赖和兼容性相关的问题):

📖 [CXL 研究思路和 Daily Arrangement 11.3 - 11.7 leanstore 配置](#)

2. cxl 的使用方式 (~~mmap~~ or 优化的 ~~exmap~~ or pread)

考虑这个问题的 Motivation: 前提是 SSD 到 CXL 的 page 粒度缓存很高效, 对应需要说明选择 I/O 模型的合理性。因为本身的想法是建立 record 粒度的 dram-cxl 交互, 但是要先证明 cxl-ssd 的页面替换路径足够高效, (dram [record] + cxl[page] > dram[page])的高效获取。中间看这个 swap 过程 IO 模型耗时主要花在哪里, 比如 pread 和 mmap 哪一个显著地好。

cidr2022 年的工作: Are You Sure You Want to Use MMAP in Your Database Management System? 否定了 mmap 的做法。

📖 [exmap 和 cxl 使用的问题](#)

根据 cidr2022 得到的结论, mmap的性能很渣, 不要用🙅。

目前的实验平台是 leanstore, leanstore 为了集成关于 exmap 的优化, (非 Linux 主分支) 需要手动编译一个内核子模块 exmap.ko 注入到当前运行的 Linux 系统中, 非常具有侵入性。

Design choice discovered in Implementation

Record 粒度缓存结构的设计

Question 1. 控制胖记录的准入阈值, 和 table 元信息

1. 在 LeanStore 读路径的日志打印过程中, 发现打印叶子页面布局时, 去掉 Page header 部分的元数据, 4096 的页面内容还剩 4064 的 Effective Page Size。作为查询之一的 tpcc customer 表记录, 由于最后一个字段是 VarChar<500> c_data, 导致完整数据的 payload = 734 B, 一个叶子节点页面存 3 个 slot 就占据 2226 B, 占到整个页面的 54.3%, 这种情况需要谨慎开启 record 粒度缓存, customer 表不太适用。
2. 如何从 page 中提取 hot record(补充冷热数据区分的相关文献)

以及 record 粒度缓存结构的设计。不管是 leanstore 还是 MySQL 都是针对不同 schema 的表单独整一个 B+Tree, 这种情况下表信息的元数据很容易区分。但是之后都提取成 record 缓存时, 混合在一起如何区分不同表, 还是依然根据不同表进行不同实例创建时的划分。

Question 2. 是否需要保留 Slot head_bytes[4] 利用 cpu cacheline locality 的设计

```
89 // -----
90 struct BTreeNode : public BTreeNodeHeader {
91     struct __attribute__((packed)) Slot {
92         // Layout: key without prefix | Payload
93         u16 offset;
94         u16 key_len;
95         u16 payload_len;
96         union {
97             HeadType head;
98             u8 head_bytes[4];
99         };
100     };
};
```

关于 union 的解释:

如果没有这个 union 字段, 每一次在页面中二分查找比较 key 的时候, 总是从 BufferFrame.dt[Slot[slot_id].offset] 的部分开始读取 key。

有了 head 之后可以先比较 Slot 中的 head, 只有当 head 相等时才会通过 offset 读取完整的 key。

head 字段对 cpu cacheline 的友好性:

直接通过 offset 拿 Key 比较会导致 Cache Miss。

比如 CPU 读取内存是按照 64 B 为单位进行读取, 所以这里

Page Header | Slot[0] | Slot[1] | Slot[2] ... | Slot[N] |

当读取 Slot[5] 时, CPU 会把附近的 Slot[4], Slot[6].. 一起拿进 L1 Cache。

Slot 数组非常紧凑, 通常只有几百 Byte, 位于同一个或相邻的几个 Cache Line 里。

Question 3. record 粒度的缓存结构采用 Hash table or B+Tree or 学习型索引?

baseline: Hash-Table

一致性问题

Question 1. Record 粒度缓存作为只读还是需要写回?

baseline: 只读

论文按照下面的章节组织:

Chapter 3. Record-granularity cache structure design and eviction policies

Chapter 4. Consistency issues(which layer to write back to and which layer to scan)

Chapter 5. CXL Memory Allocation and tail-latency breakdown analysis

Chapter 6. Experimental comparison with Tiered-Indexing

Chapter 3. Record-granularity cache structure design and eviction policies

这是 RecordCache 的完整设计文档, 将会用

🤔 标记所有设计过程中的决策和 Trade-Off,

❗ 表示当前设计是可以返工修改的,

🌟 表示有强烈的依据选择当前的设计而不是其他。

RecordCache 是 位于 TPCCWorkload 和 BTree Index 之间的一层 Tuple-Granularity Cache。

核心是完成:

TPCCWorkLoad -> RecordCache -> B+ Tree Index Pages -> CXL Page-Sized Buffer Pool

(较高延迟, 高带宽: 高带宽带来的好处是在 Slow Path 上, 另一个模块 CXLPagesPrefetch BackgroundThread 可以比较激进地做预取, 3.2 节讨论)。

的交互流程, 以及 4 个核心操作 lookup, scan, insert, update 的状态转换图和执行流程, 详细描述 4 种操作过程中的一致性处理。

通过缓存 Hot Record 来实现 "CXL-Enabled Two-Way Acceleration"

1. Fast Path: 大部分的 lookup, update 由 recordCache 完成, 绕过 BTree 遍历, 大量具有倾斜性质的工作负载访问可以直接命中 RecordCache, 节省 DRAM 资源。(否则大量 Cold Page 中少数 Hot Record 的访问将占据有限的 DRAM 内存资源)。

一旦把 Page 中的 hot record 提取到 recordCache 中, 上层 record 访问直接命中 recordCache, 不会穿透底层的 Page, 此时会驱逐 Cold Page, 但仍然需要保留 hot record。Cold Page 和 Hot record 的生命周期是冲突的。

2. Slow Path: 大部分的 record miss 会由 CXLPages Cache 命中, 减少从 SSD 加载的 IO 开销延迟。

1. Structure Design Rationale

🤔1. Inclusive or Exclusive or Hybrid(Non-Inclusive)

RecordCache 中缓存的内容是 CXLPagesCache 的子集, 还是和 CXLPagesCache 彼此独立, 不包含?

方案	问题
Inclusive	1.反向索引维护开销 当 CXL Page-Based BP 驱逐当前页面的时候, 需要反向查找当前 page_id 对应的所有 Record, 进行反向索引的 Invalidate 通知, 需要维护 page_id -> {record_id_1, record_id_2, ..., record_id_n} 的映射关系。❌
	2.连带驱逐问题 Record 访问发生在上层 recordCache, 不会再增加 CXLPagesCache 中的访问计数, 驱逐 Cold Page X 会导致 RecordCache 中的 Hot Record Y 被连带驱逐, 因为 Y 的访问计数不会穿透到下层 CXLPagesCache ❌

Exclusive	页面删除的开销 从 Page-Based BP 中提升 Record, 需要处理 BTreeNode 的 record 删除空洞 ❌
Non-Inclusive 🌟	Page eviction 不影响 record eviction, 同时 record promotion 操作不会变成 B+ 树叶节点的split/merge 操作。

🤔2. RecordCacheEntry 应该存完整的数据，还是只存指针？

方案	问题
Inclusive 模型存指针	指针访问引入 CXL 路径访问的延迟 ❌
Exclusive 模型存数据	模型本身的页面删除开销 ❌
Non-Inclusive 模型存数据 🌟	无 CXL 访问延迟， 无连带驱逐、删除问题

🤔3. 所有权模型设计(写权限管理)

🌟: 当前设计决策:

Non-inclusive + 单一写所有权(当 Record 同时存在 RecordCache 和 PageCache 中, RecordCache 具有优先的写权限)

recordCache 拥有 record 的优先级写权限, 原因是 CXL 的写延迟相较于 DRAM 仍然较高, ([1] Chapter 6.Table 2. 分析了在本地 DRAM 访问延迟的基础上(< 100 ns), 访问 CXL 互联的 DRAM 设备带来的 CXL 协议栈固有延迟).

端到端系统访问的延迟包括: CXL 协议栈延迟 + 链路 round-trips + Retimer 延迟 + 设备内部处理延迟。

在不增加 CXL Switch 的设计下, CPU side load-to-use: < 100ns(include DRAM access) + CXL stack round-trip of 50ns + 10ns flight time + 10ns Retimer round-trip = 170ns)

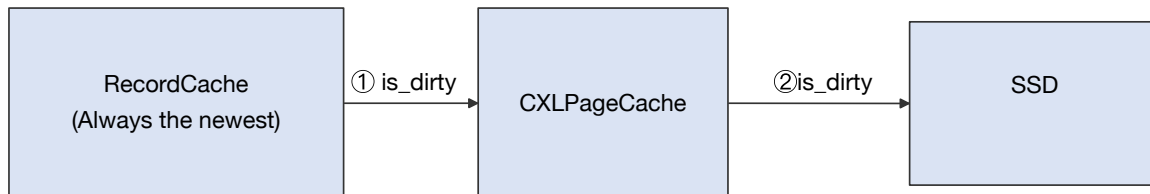
RecordCache 具有优先级的写权限可以尽可能缓解额外 70 ns 写延迟带来的 OLTP 性能下降。

模型属于 Non-inclusive (系统中存在某一时刻 recordCache 和 PageCache 中拥有相同记录), 也存在 record 在 PageCache 中已经消失, 但 recordCache 中依然维护。

设计原则: 如果 recordCache 中存在 record, 那么它是最新的。RecordCache 在所有权模型中是"一等公民"。

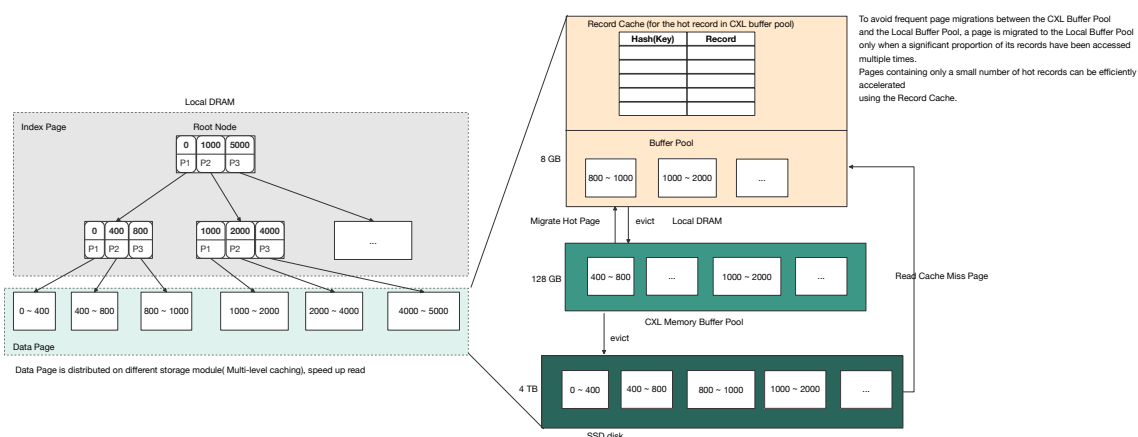
在上层 RecordCache 访问命中绝大多数 record 之后，读 record 的计数不会再算进 CXLPageCache 中 Page 访问的统计，record 和 Page 的驱逐是独立的。

跨存储层次一致性的问题时，recordCache 只负责和 CXLPageCache 的同步，持久化逻辑由CXLPageCache 负责。



🤔4. 关于RecordCache读写缓存的定位:

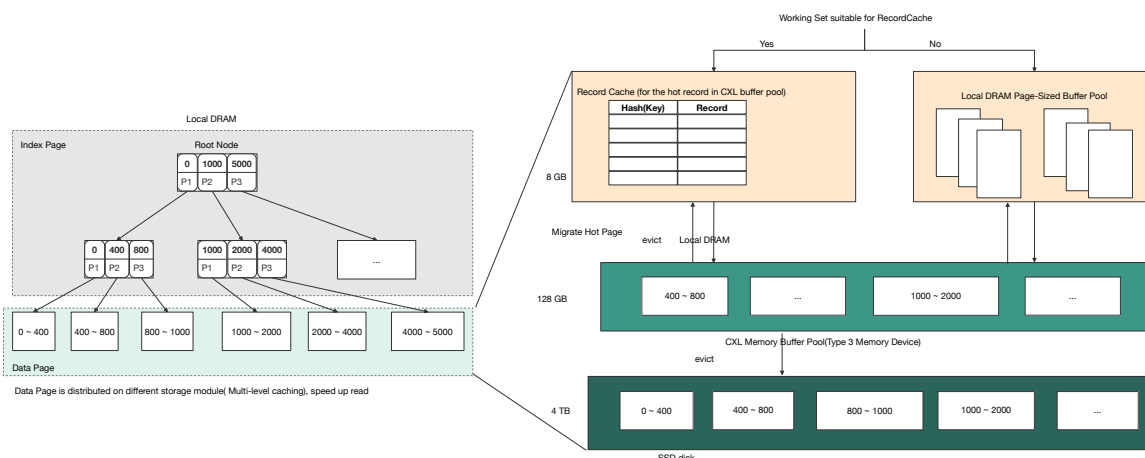
1. 作为只读缓存的系统结构图:



2. 选择作为读写缓存:

原因: 不适合 RecordCache 的负载，依然通过 DRAM Page-Sized Buffer Pool 完成读写。适合 RecordCache 的负载支持读写，可以减少延迟造成的性能下降，同时符合节省空间的需求

要。



🤔5. RecordCacheEntry 结构设计

- page_id
- key, value 的数据副本
- is_dirty

dirty eviction 时需要 merge record 到 CXLPagesCache 中，因此维护 page_id 和 is_dirty 协同操作

2. Operation Flow Design Rationale

本节描述 RecordCache 加速的 lookup 和 update 工作流程，为了满足一致性的要求，scan 和 insert 也会不断访问 recordCache 来确保下层 B+Tree 中的数据是最新版本。

第 2 节中展示的是验证符合倾斜工作负载，开启 recordCache 的执行路径。对于运行中不需要 recordCache 的路径，走一般的 DRAM Page Cache 流程，在 3. eviction 节中对这部分驱逐逻辑进行说明，Interaction Flows 不涉及。

下面 4 张执行流程图中，黑色部分为普通的执行逻辑，蓝色部分为 RecordCache 引入的一致性检测和 record 资源管理流程，红色部分为加速的 Fast-Path。

2.1 Lookup Flow

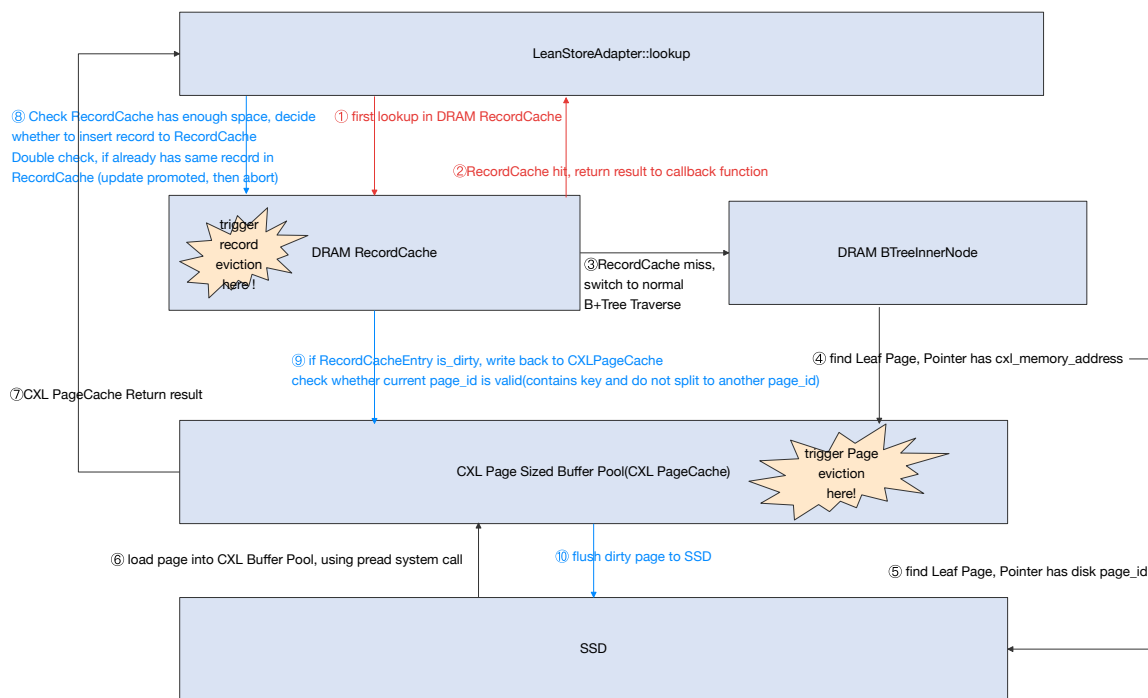
🤔 Lookup 1: Adapter 层额外决定 Lookup 是否写入 RecordCache (同步 vs 异步)

同步的问题在于引入更高的尾延迟, 如果遇到

Record write -> RecordCache eviction -> is_dirty = true -> Unswizzling(page_id) -> pread page to CXLPageCache -> merge record -> flush to SSD

造成较高的长尾延迟

先实现同步, 可以优化成异步 Ring Buffer, Adapter层负责把 record 写入 RingBuffer, 后台线程处理队列消耗)。



①(Fast-Path): Adapter 层访问 RecordCache, 查询相关记录

②(Fast-Path): RecordCache 中存在对应记录, 可以直接返回

③ 在 RecordCache 中没有找到记录, 走 DRAM 中的 B+树索引进行遍历

④ 定位到叶子页面, 在内存中可以直接通过 cxi_memory_address 访问

⑤定位到叶子页面, 不在内存中, 通过磁盘 page_id 进行读取

⑥把页面从磁盘加载到 CXL PageCache 中

⑦ 返回 CXL PageCache 中查询到的结果

⑧ 判断当前 RecordCache 中是否有足够的空间，判断本次是否需要把 Record 插入 RecordCache，同时需要再次检查 record 是否位于 recordCacheEntry 中，可能会有 update 线程也在执行 promote 操作。此时无条件 abort，因为 update 的 record 总是比 lookup 得到的更新。

⑨ (后台线程): Lookup 线程会触发 Record Eviction，如果发现对应的 RecordEntry 标记为 is_dirty = true，写回到 CXLPageCache，并检查当前 page_id 是否有效。（eviction过程中没有出现因为 insert 导致的页面分裂而导致 page_id 失效）

⑩(后台线程): Lookup 线程会触发 Page Eviction, 处理刷脏页到磁盘的过程。

在图上没有展现出来的细节，当 ⑨ 中触发 RecordCacheEntry is_dirty 的逻辑时，CXLPageCache 中对应的 Page 有可能已经被驱逐回磁盘了。但 RecordCacheEntry 记录了 page_id, 需要把 page_id 的页面再次读回来，再写盘。同时可能因为 insert 导致原先的 page_id 分裂成新的 new_page_id, 在读回 page 之后判断当前 recordCacheEntry 中的 key 是否在范围内。

⑨ If RecordCacheEntry is_dirty, write back to CXLPageCache

同时确保 page_id 对应的页面在 CXLPageCache 中，否则触发额外的

Read page_id to CXLPageCache -> Merge record to page -> flush to SSD.

目前 Adapter 层的 write-back Record to RecordCache 是同步的，后续改成异步的 Async Write Buffer。

其中 Performance 的部分由红色展示，兼容的基础情况由黑色展示，is_dirty 的逻辑和写回过程(record 和 page 资源的管理和释放)由蓝色展示。

🤔Lookup 2: Lookup 路径上，dirty record 写回需要对应 page_id 的读盘操作(Read-modify-write),eviction 不完全独立

Page 和 record 独立驱逐的问题在于 5 中描述的 dirty merge 时需要读 Page，

引入 Async WriteBuffer 之后根据尾延迟的情况决定是否增加 pin CXLPageCache 中 page 的设计。

🤔Lookup 3: lookup 和 update 的一致性问题

线程 A 执行 lookup，发现 RecordCache miss, 从 CXLPageCache 中读出对应的 Record v1 版本写回到 RecordCache。

线程 B 执行 update，发现 RecordCache miss, 从 CXLPageCache 中修改 Record v2 版本后也 promote 到 RecordCache。

RecordCache 中新的版本是 Record v2，但是 lookup 返回 Record v1。

解决思路：

状态锁表：

promote 阶段类似 pread page_id 的过程，leanstore 中设计一个 io_ht 表项，表示当前是否有线程正在读取或者 promote。

promote 部分的逻辑单独设计成一个结构体，这个结构体有对应的状态，比如 Reading, Ready, Free 等。

当 lookup 线程首先拿到对应 record 的 promote 结构体时，设置 Reading 状态，update 线程阻塞等待。（✗）

写优先读：

状态锁表的开销大，这里准备每次当 lookup 读入 record，但发现 recordCache 中已经有了另一版本的数据，那么总是放弃本次读取，重新执行。update 意味着 record 的来源总是更新，update 优先 lookup 的执行。

2.2 Scan Flow

🤔scan 1:scan 部分是否需要处理 dirty record 的写回逻辑

从一致性的角度考虑，每次 scan 得到 record 结果，都走一遍 lookup 去查询 RecordCache 中是否有相同 key 的，标记为 dirty 的 record，发现 dirty 则触发 merge record 的逻辑。

从性能的角度考虑，scan 从一个读操作变成 record 刷脏的写操作，过程中读锁升级为独占写锁。

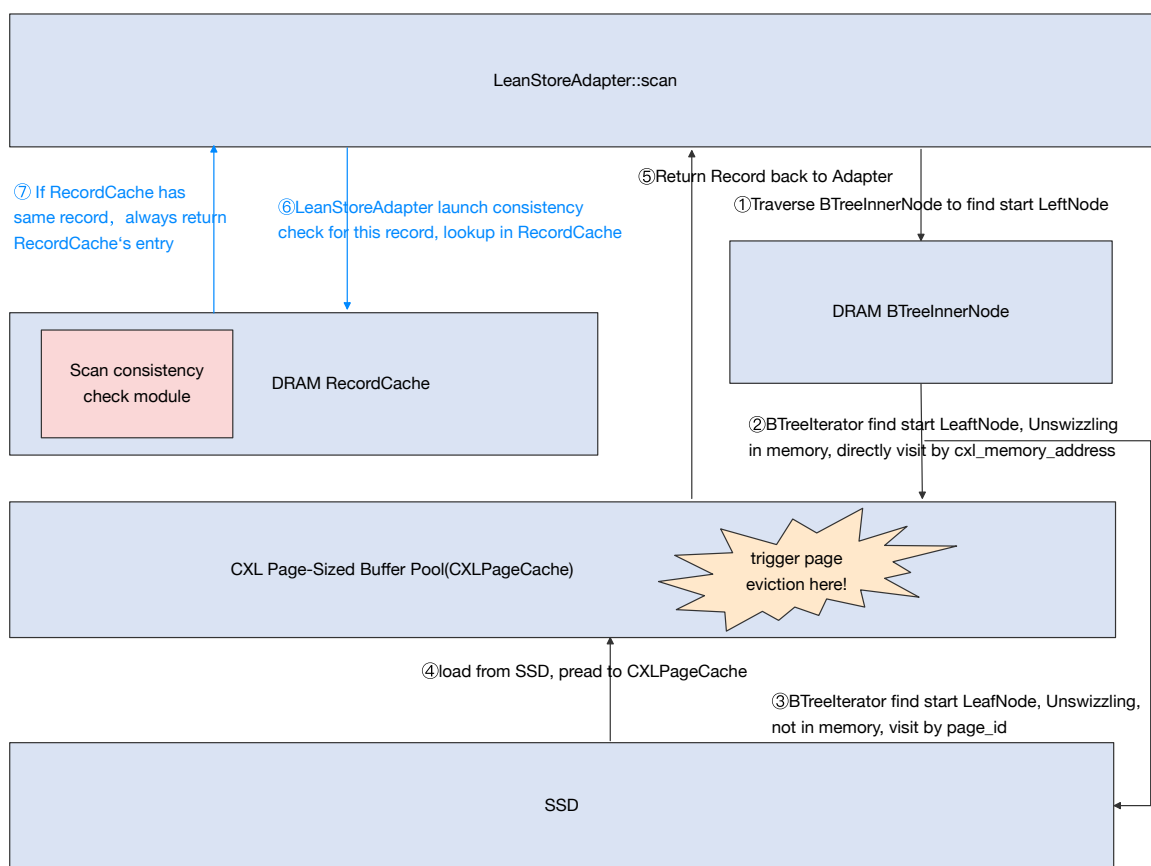
- 方案一 On-demand write-back scan(✗ 正确，但性能极差)

scan 正常执行，从 CXLPageCache 中读出数据后，每次都由 Adapter 层发起对 record 是否 dirty 的检测，走一遍 recordCache 的检查，如果 RecordCache 中存在 dirty record，找到对应的 BTreeLeafNode，触发 merge record 的过程。

同时 scan 部分把读锁升级成写锁的过程，容易阻塞其他写线程的操作。merge 的部分必须异步处理。

- 方案二 Read-Only

scan 部分不负责 recordCache 的 merge dirty record，一旦发现 RecordCache 中有相同的记录，直接返回。



① Adapter 层发起 Scan 操作，遍历 BTreeInnerNode 找到起点的 LeafPage

② LeafPage 在内存中，可以直接通过 cxl_memory_address 访问

③ LeafPage 在磁盘上，通过 page_id 定位磁盘块

④ 从磁盘加载对应页面到 CXLPageCache

⑤ 返回查询结果给 Adapter 层

⑥(一致性检查): 判断当前 Scan 的 key range 是否在 RecordCache 中有记录

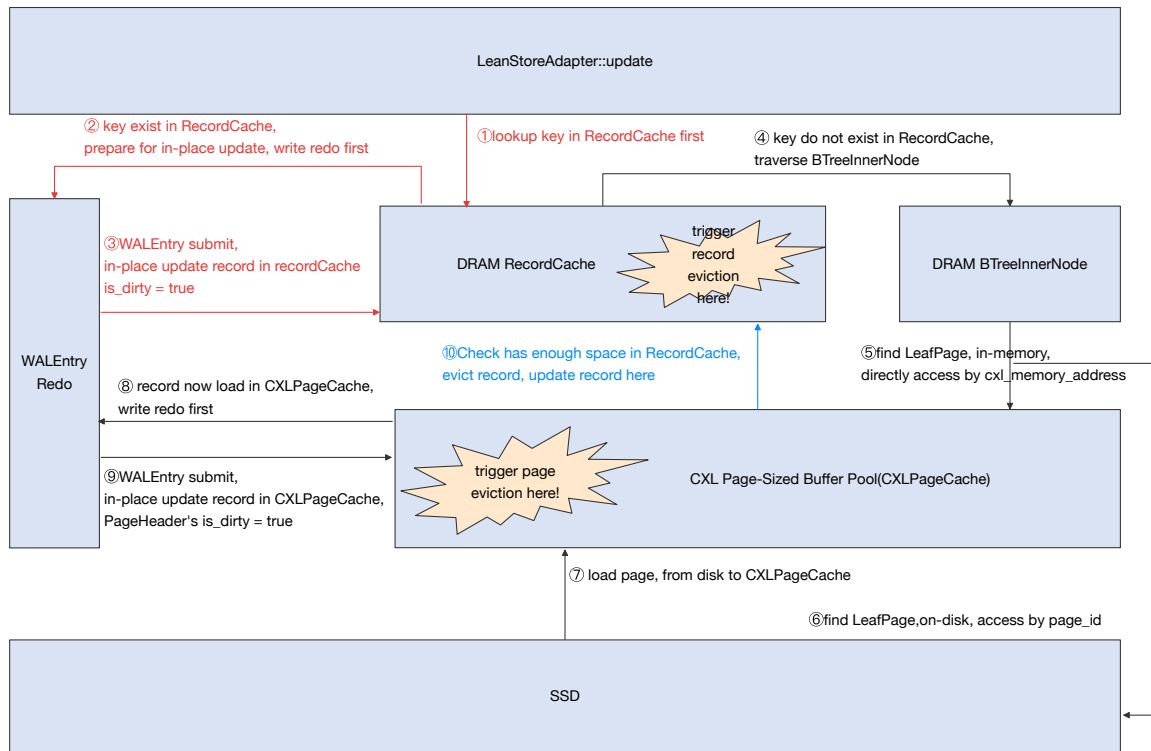
⑦(一致性检查): 如果在 RecordCache 中找到记录，直接返回 RecordCache 中的结果。

这里不处理 Record is_dirty = true 的 merge record 逻辑，等待后台线程 evict record 的时候异步触发。

Scan 操作的占比很少(倾斜工作负载情况下)，可以忍受 CXL 读延迟以及需要二次查询 RecordCache 的性能开销。

可以做的优化: 改造 CXLPagesCache 中 Page 结构的 PageHeader 字段, 当 Update record 时设置 PageHeader 的 has_dirty_record = true, 否则可以直接跳过 ⑥⑦。

2.3 Update Flow



- ①(Fast-Path): Adapter 层发起 update 执行, 首先查询 RecordCache 中是否有相关的 key
- ②(Fast-Path): RecordCache 中存在对应的 key, 定位到符合条件的记录, 写 redo
- ③ (Fast-Path): 写完 redo 之后就地更新 RecordCache, 返回执行结果。
- ④ RecordCache 中不存在相关记录, 遍历 BTreeInnerNode 定位到 LeafPage
- ⑤ LeafPage 在内存中, 可以直接通过 cxl_memory_address 访问
- ⑥ LeafPage 在磁盘上, 通过 page_id 找到
- ⑦ 把页面从磁盘加载到 CXLPagesCache
- ⑧ 执行 update 操作之前, 写 redo

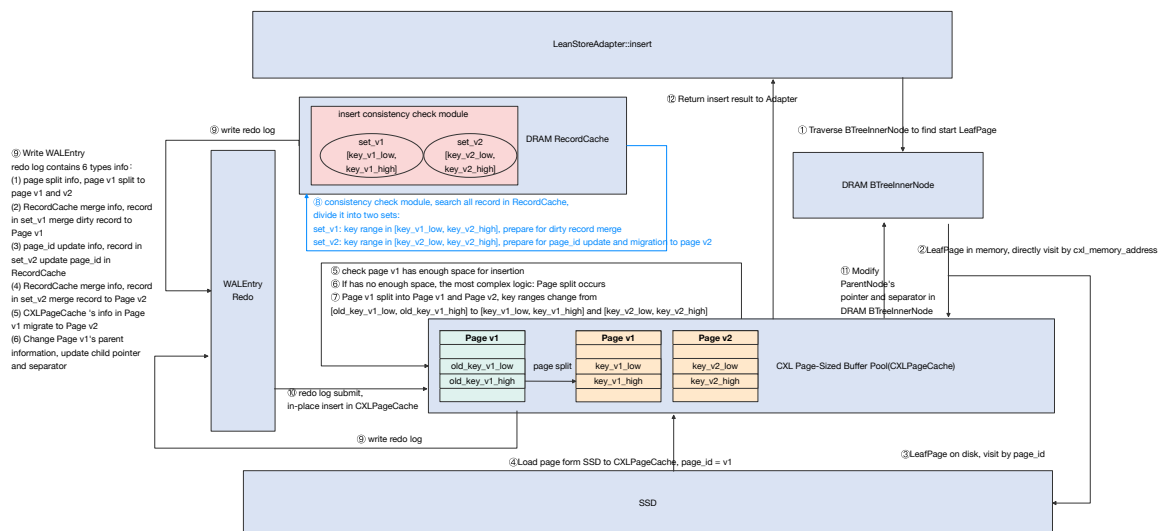
⑨ redo 提交, 在 CXLPageCache 中执行就地更改, PageHeader is_dirty = true

⑩(promote 逻辑)处理 promote 的情况, Update 的优先级总是高于 Lookup,

默认策略是, 写完 CXLPageCache 之后, 总是把对应的 Record 写入 RecordCache,采用 On-demand record promotion on update.

(需要经历和 lookup 阶段相同的 RecordCache record eviction 和 dirty record merge 流程)。

2.4 Insert Flow



insert 阶段的执行流程:

考虑最复杂的情况, 当 Adapter 层触发 insert 逻辑时, 遍历 B+Tree 后, 发现原先的 Page v1 空间不够, 需要产生页面分裂和记录迁移。现在 Page v1 分裂成 Page v1 和 Page v2。

1. 对于 Page v1(旧页面): 需要检查 RecordCache 中属于 page_id = v1 的 dirty record merge 到 Page v1 中。

2. 对于 Page v2(新页面): RecordCache 中原本属于 page_id = v1 , 但随着页面分裂后, key 范围属于 page_id = v2 的缓存项, 必须修改 page_id 信息。

执行过程如下:

- ① Adapter 层发起 insert 请求, 遍历 BTreeInnerNode 找到起始的 LeafPage
- ② 判断 LeafPage 是否在内存中, 在内存中可以直接由 cxl_memory_address 访问
- ③ 判断 LeafPage 不在内存中, 需要根据 page_id 从磁盘中加载页面到 CXLPagesCache。
- ④ 从磁盘中加载页面到 CXLPagesCache, 后台线程负责 CXLPagesCache 中 page eviction, 不在 insert 执行的关键路径上。
- ⑤ 现在确定页面在 CXLPagesCache 中, 记录 page_id = v1, 判断当前 LeafPage 上是否有足够的 slots 空间来插入当前记录。
- ⑥ 判断 Page v1 上空间不足, 需要发生页面分裂, 由分配器得到新页面的 page_id = v2, 在 CXLPagesCache 的 free_list 上申请空白页面作为新页面 Page v2。
- ⑦ 重新分配 key, 原先 Page v1 上的 key 范围是 [old_key_v1_low, old_key_v1_high], 现在 Page v1 负责 [key_v1_low, key_v1_high], Page v2 负责 [key_v2_low, key_v2_high]。
- ⑧(一致性检查模块): 设置两个集合 set_v1 和 set_v2, 查找 RecordCache 中 key 的范围落在 [key_v1_low, key_v2_high] 的 record, 以及查找 RecordCache 中 key 的范围落在 [key_v2_low, key_v2_high] 范围的 record。

(一致性检查模块): 如果 set_v1 不为空, 检查是否需要把 dirty record merge 到 CXLPagesCache 的 Page v1。

(一致性检查模块): 如果 set_v2 不为空, 这些 record 需要更新自身的 page_id, 同时也需要合并到 CXLPagesCache 的 Page v2。

⑨(一致性检查模块): 写 redo log, WALEntry 需要记录下面 6 类 redo 信息:

- (1) CXLPagesCache 中的 page v1 发生分裂, 分裂成 page v1 和 page v2
- (2) RecordCache 中属于现在 page v1 key 范围的 dirty record 合并到 CXLPagesCache 中的 Page v1
- (3) RecordCache 中属于现在 page v2 key 范围的 record 信息 update, page_id = v2
- (4) RecordCache 中属于现在 page v2 key 范围的 record 信息合并到 CXLPagesCache 中的 Page v2
- (5) CXLPagesCache 中 Page v1 的部分记录迁移到 Page v2
- (6) 修改 Page v1 的父节点信息, 更新 child 指针指向和 separator

⑩redo log 提交, 在 CXLPagesCache 中执行就地更改

⑪对应修改 DRAM BTreeInnerNode 中 Page v1 对应的 Parent Node, 修改 Child 指针和 separator

⑫返回 insert 执行结果给 Adapter 层。

3. Lifecycle & Corner Case

🤔 Lifecycle: Stale page ID 的问题(一致性问题: insert 和 recordCache page_id 的一致性维护)

假设在 RecordCache 中维护了 Record A, Record A 对应的 page_id = P1

随着后续大量的 insert 操作, 导致 P1 页面产生分裂, 分裂成 P1 和 P2, Record A 相应从 Page P1 移动到 Page P2, 后续 dirty evict 的时候如果把 Record A merge 到 Page P2 会产生错误。

解决思路: 当 dirty evict 需要 CXLPageCache merge Record 的时候, 触发检测 page_id 是否有效的逻辑。(后台处理, 设置成 Async writeBuffer)这里检测的时间不计入关键执行路径。

由于 RecordCache 中存在完整的 record_key, 在执行 dirty merge 的时候, 先检查当前 page_id 对应页面的 key 范围, 如果 key 范围并不包含 RecordCache 的 record_key, 说明发生过页面分裂和记录迁移, 此时需要执行 lookup 重新查找 page_id。

```
if(lookup_key_for_page_id(RecordCacheEntry record) == record.page_id)
```

3.1 Promote

作为后台处理线程, 专门负责把 CXLPageCache 中的 hot record 提升到 RecordCache 中。

Record 共有两次 promote 的时机:

1. Lookup 阶段查询过程中, 选择

- a.需要改造 CXLPageCache 的 Page 结构, 每个 Page 额外维护 slot_id 不同访问次数, 根据统计信息提升 hot record
- b.牺牲一部分准确性, 不改造 Page 结构, 采用概率方式, 每次 lookup 的记录有 10% 的概率被 promote 到 RecordCache。

(a 和 b 的 trade-off)

2. Update 阶段更新过程中, 选择激进的 promote 策略

一旦执行 Update, 就必须 promote 对应的记录到 RecordCache, 在 RecordCache 中做修改。

3.2 Eviction

RecordCache 的驱逐逻辑和 CXLPageCache 独立, 区别在于当 RecordCacheEntry 的 is_dirty = true 时, 需要先合并 dirty record, 再刷脏落盘。此时 page_id 的信息可能过期, 同时对应的 page 可能不在内存中, 造成较长的延迟。

Eviction 部分对 RecordCacheEntry 结构提出的设计要求:

RecordCache 需要有给定 page_id, 快速索引到 CXLPagesCache 中对应页面的结构。

dirty evict

合并到 PageProviderThread 线程的驱逐逻辑中, 对 dirty record 也维护 Async WriteBuffer 等待最终合并到内存页面和落盘。

目前 RecordCacheEntry 中维护了访问次数的统计信息, 根据访问频率划分 Hot - Warm - Cold 之后进行驱逐。定期扫描删除访问次数低, 非 dirty 的记录。

3.3 WALEntry Recovery

🤔 Recovery 的逻辑

WALEntry 在原本的逻辑是重放恢复 Page, 那么加入 RecordCache 之后如何重放。

解决思路: Recovery 只恢复 CXLPagesCache, 不处理 RecordCache。每次 Recovery 阶段, 默认 RecordCache 重启后设置为空。

Recovery 不做特别设计, 这里默认 WALEntry 只恢复 CXLPagesCache, 可能的情况是直接恢复 Page id 对应的信息, 或者由 RecordCache 写过程中产生的 WALEntry, 但本身不用来恢复 RecordCache。

默认重启后, RecordCache 设置为空。

3.4 Type 3 HDM

🤔 CXL Buffer Pool 部分的数据访问不进入 Host cpu cache

好处是:

1. Host cpu cache 中缓存 RecordCache 里的数据, 避免缓存污染
2. 如果 CXL Buffer Pool 中的 record 进入 Host cpu cache, 在驱逐的时候发现对应的是 Host-Managed Device Memory(Type-3 设备内存), 会把这部分驱逐的请求打包成 CXL flits, 有一个 CXL 协议转换的开销在。

关于 cpu cache 是否能够快速缓存最热的数据:

硬件是组相连的设计, 一次 scan 会刷掉一个 way 的数据,

软件上可以做全相连。

如果两个应用同时跑, 一个局部性很好, 另一个是流式处理数据没有局部性, 如果共享 cache, 性能会大幅度下降, 所以需要分隔开来。[2] (问题在于单纯减少命中率并不能反映

出系统运行的很好，[2] 的 Figure 1 展示的 P 类并行访问序列和 S 类孤立访问序列，LRU 算法驱逐最不常访问的组，但造成更多的 memory stall)

4. Future Optimizations(Planned)

Replace sync RecordCache write with Ring Buffer(Background merge)

5. Remark

第一版设计的反馈意见:

1. RecordCache 作为读写缓存的 Motivation 不充分，原因是写操作的 redo log 延迟远远高于写 CXL 相较写 DRAM 的延迟。为了减少写延迟而把 update 放在 RecordCache 的理由不成立。
 2. RecordCache 是已经 solved 的问题，没描述清楚挑战在哪里。
 3. cpu cache 的情况很复杂，不只是 scan,lookup 进入，还有 query optimizer 和众多后台线程，不让 cxl type 3 设备上的内容进入 cpu cache 的动机不够。
 4. scan 操作的设计非常不好，加 bloom filter 也不行，因为 bloom filter 相对静态，不支持频繁 update 的 RecordCache 的 bloom filter。
 5. RecordCache 和 BP 之间的逐出，替换问题加上 ai 相关的内容新颖性会更高
 6. RecordCache 采用 write-back 还是 write-through, 需要实验观察下对读写以及 scan 的具体影响，有数据支撑，论文整体逻辑会更严密。
-

重构版本

1. Design Rationale

RecordCache 作为读写缓存的意义:

1. RecordCache 本身提供了 Tuple 级别的并发，更新 RecordCache 时不需要锁定底层的整个页面，在 RecordCache 结构上能够提升吞吐量(Throughput)，解决了 Page 级别锁竞争的瓶颈。
2. 对 CXL 的写合并优化(Write Coalescing)

如果一个热点记录被更新 10 次，底层 Page 可能会被标记为脏页 10 次。

启用 RecordCache 之后，10 次更新都发生在 DRAM RecordCache，在 eviction 时才合并成一次 CXL 写操作传回页面，节省不必要的链路带宽浪费。

2. Challenge

1. RecordCache 和 Page-Sized Buffer Pool 的大小比例

RecordCache 的容量并不是越大越好，在命中率和吞吐上有一个性能拐点，识别这个性能拐点。

2. 跨存储层次的一致性保证

3. B-Tree insertion 导致的 page_id 失效问题

4. Hot record 和 cold page 生命周期冲突的问题

3. Operation Flow

为了更准确地验证 RecordCache 作为 write-through 和 write-back 两种策略下的性能优化决策，

设计阶段实现两套执行逻辑，write-through 和 write-back，分别比较读写，scan 操作的性能影响。

3.1 Write-through

Update 操作同时修改 RecordCache 和 CXL Page，写 WAL

一致性能够保证，scan 不需要检查 RecordCache 脏数据

3.2 Write-back

Update 操作只修改 RecordCache，标记 is_dirty，只有在 Eviction 时才合并回 Page。

好处是 Write Coalescing（写合并）。

这里对 Write-through 和 Write-back 做一次说明：

由于需要实验数据支撑，在实现了 DRAM RecordCache 具有读写缓存之后，和 CXL BP 的写采用 write-through 还是 write-back 的写回方式。

不同的写回方式对应不同的数据模型，

write-through 要求，一旦更新在 RecordCache 中的 record 记录，就同时更新 CXL BP 中的记录，

可以推导得出，RecordCache 一定是 CXL BP 数据的子集，eviction 的部分不能独立驱逐 (RecordCache 可以驱逐 Record，但是不能存在 RecordCache 中还有记录，但是 CXL BP 中没有)，必须对应 Inclusive 的模型。

write-back 要求，可以合并多次 RecordCache 的写操作，在 dirty evict 或者强制需要一致性的时候再和 CXL BP 同步，可以单独驱逐 CXL BP，在 RecordCache 合并记录的时候，再从 SSD 中读取页面进行合并，不一定必须是 Inclusive 的模型，可以是 Non-Inclusive 的模型。

参考资料:

[1] An Introduction to the Compute Express Link(CXL) Interconnect. Debendra Das Sharma, Robert Blankenship, Daniel S. Berger (2023, reference 144).

[2] A Case for MLP-Aware Cache Replacement(2006, ISCA)

Chapter 4. Consistency issues(which layer to write back to and which layer to scan)

Chapter 5. CXL Memory Allocation and tail-latency breakdown analysis

Chapter 6. Experimental comparison with Tiered-Indexing

Experiments

Motivation Part Experiments

为了验证整个工作的 Motivation，我们做了下面的实验来说明工作的合理性。

分层内存系统对不同工作负载的增益情况实验

测试了 Buffer Pool Size = 2 G / 4 G / 8 G / 16 G，对 Sysbench 中数据分布为 zipfian/pareto/uniform

展现出扩展内存容量对于 uniform 的数据分布增益有限，因此在引入 cxi 作为新一级内存层次之后，focus 的重点是倾斜工作负载。

 [polardb-mysql tp 内存组件迁移实验BP\(草稿版本 2025/10/10\)](#)

 [polardb-mysql tp 内存组件迁移实验BP\(草稿版本 2025/9/22\)](#)

page 访问过程中 record 占比情况实验

不局限于数据库系统，当前分层内存系统中，普遍存在使用大页面来降低地址转换开销的工程上 trade-off。但问题的本质在于访问不均匀性，一个大页面内的各个子页面访问频率存在显著差异。Memtis 在内存密集型基准测试程序 Silo 中发现，对于 2MB 的大页(包含 512 个 4MB 的子页)[0 - 512]，绝大多数大页中只有不到 100 个子页被访问。(Not all subpages in a huge page are equally hot).

相比于 os 层面的 Huge Page，database 层面同样存在 record 与 page 粒度之间的问题。我们测试 sysbench vs TPCC 工作负载中单次页面请求和具体涉及到的 record 占据内存空间的比例。进一步量化 DRAM 内存空间浪费的程度。

理论上最高带宽利用率公式计算的问题

```
[zhizhi.tyf@k69d03146.eu95sqa /home/zhizhi.tyf/cxl_design_and_implementation/leanstore/python-scripts]
$python3 analysis_5th.py /home/zhizhi.tyf/cxl_design_and_implementation/leanstore/testing_log/tpcc_run_5th.log
=====
LeanStore Log Analysis: /home/zhizhi.tyf/cxl_design_and_implementation/leanstore/testing_log/tpcc_run_5th.log
=====

1. Hierarchy Access Statistics:
=====
Level      | Type      | Unique Pages | Total Accesses
=====
Level 0    | Root      | 1            | 120
Level 1    | Inner     | 28           | 120
Level 2    | Inner     | 119          | 120
Level 3    | Leaf (Approx) | 120         | 120
=====

2. Disk IO Statistics:
=====
Unique Pages Evicted (Working Set IO): 87
Total IO Events (Approx. Cache Misses): 103
Overall Cache Miss Rate (Est.): 85.83%

3. Bandwidth Efficiency Analysis:
=====
Total Record Data Needed: 0.09 MB (90,035 bytes)
Total Page Data Loaded: 0.47 MB (495,616 bytes)
Effective Data Ratio: 18.17%
Bandwidth Waste: 81.83%
Potential Gain with Record Cache: 5.5x
=====
```

现在想估算一个 record 粒度缓存在理论上(假设每一次访问都直接在 record 粒度缓存中直接命中)，这种最好情况下能够节约的内存容量的理论上限。

跑了一个 tpc-c 的 orderStatusId 类型的 customer.lookup 的单点查询。以这一次查询的数据为例，

根据第 5 层 findLeafCanJump 的打印结果，判断一共发起了 120 次查询，其中 120 次查询涉及到 Level 0 (根节点) 1 个页面，Level 1 (内部节点) 28 个不同的页面，Level 2 (内部节点) 119 个不同的页面，Level 3 (叶子节点) 120 个不同的页面。

其中 Level 3 的随机叶子页面一共发起了 103 次 IO，涉及到 87 个不同的页面。

(1) record 粒度缓存是否只针对最后一层随机叶子页面进行。

目前判断 Level 0 和 Level 1 的访问相对集中，Level 2 的访问相对分散。

但这里内部节点都是紧凑排列的 key，感觉这里做 record 粒度缓存收益不大

(2) record memory capacity / total page capacity 的理论收益公式

(2.1) 第一种计算方法是直接看叶子节点里 record 的排列, 其中每个 customer 字段的 payload 长度是 733, key 的总长度是 12, 列出共同前缀之后, key_length 长度为 1。在每个叶子页面中, 一共存储 3 个 customer 数据, 打印出 record 的 Space Used = 2226 B, 整个 4096 页面中有效的数据空间是 4064 B

$745 \text{ B} / 4096 \text{ B} = 18.18\%$, 节省的带宽约为 81.82%

(2.2) 第二种计算方法是 120 次查询访问中, 一共发起了 103 次 IO, 计算实际需要的 record 记录 $(120 * \text{customer payload_length}) / 103 * \text{PageSize} = 120 * 745 \text{ B} / 103 * 4096 \text{ B} = 89400 / 421888 = 21.19\%$

节省的带宽约为 78.8%

(2.3) 第三种计算方法是带上前 3 层的容量收益

Page 粒度缓存:

$\text{total_memory_page_size} = \text{Level_0_page} + \text{Level_1_page} + \text{Level_2_page} + \text{Level_3_page}$
 $= (1 + 28 + 119 + 120) * 4096 = 1097728 \text{ B}$

Record 粒度缓存:

$\text{total_memory_record_page_size} = \text{Level_0_page} + \text{Level_1_page} + \text{Level_2_page} + \text{Level_3_record}$
 $= (1 + 28 + 119) * 4096 + 120 * 745 = 695608 \text{ B}$

$\text{total_memory_record_page_size} / \text{total_memory_page_size} = 63.37\%$

节省的带宽约为 36.63%

反馈:

几种方式都有问题。不论是 record cache 还是 page cache, 第一次读总是要从磁盘上加载整个 page。如果没有 page 缓存则每次读 page 都要 io。

如果有 page 缓存则可能命中缓存。

考虑 record cache 不能只考虑 record 本身, 实现 record cache 的辅助结构也要考虑。

比如用 LRU 记录每个 record 的开销就很大。

如果专门用一棵 in-memory B+tree 记录 hot record, 则读操作本身也涉及到 hot B+tree 的 purge 和 insert, 也就是写操作。

写操作又进一步涉及到 record move, page split merge 等。

消耗的 CPU 开销也不小。

如果只是考虑内存开销, 则应该用相同的内存。分别实现 hot B+tree 和 page cache, 看下谁的性能高, 或者看下谁的 IO 更小。

CXL SSD Page 粒度IO 效率实验

由于我们的整个设计基于这样的假设

$\{\text{dram-cxl}\}[\text{record} + \text{page}] + \{\text{cxl-ssd}\}[\text{page}] > \{\text{dram-ssd}\}[\text{page}]$

而这个假设需要证明的一个重要观点就是: {cxl-ssd} 的 Page 粒度IO 效率足够高。

这些我们已经有了有一些依据。比如在 Unlocking the potential of CXL-Switch 的工作中，验证把整个 Buffer Pool 移动到带有 Switch 的 CXL-attached memory 上性能下降在可接受的范围内。

本质上，cxl 是一个cpu 和内存交互的协议，而 cxl 设备的 io 过程实际上和普通的 DRAM IO 类似，都是由 cpu 发出 pread 指令，进行文件系统的加载，但需要增加额外的 cxl 协议延迟。

同时由于增加了一层额外的缓存层，相比之前 page cache miss 之后从 SSD 加载，现在需要经过两层 record cache miss, 和 page cache miss, 需要说明引入 record 粒度缓存之后的尾延迟可控。

参考文献

- [1] Rearchitecting the Thread Model of In-Memory Key-Value Store with uTPS(SOSP2025)
- [2] Tiered-Indexing: Optimizing Access Methods for Skew(PVLDBJ2025)
- [3] HybridTier: an Adaptive and Lightweight CXL-Memory Tiering System(ASPLOS2025)
- [4] HiDPU: A DPU-Oriented Hybrid Indexing Scheme for Disaggregated Storage Systems(FAST2025)
- [5] Memtis: Efficient Memory Tiering with Dynamic Page Classification and Page Size Determination(SOSP 2023)
- [6] The Case for Learned Index Structures(SIGMOD2018)